

PRODUCT REQUIREMENTS DOCUMENT

(PRD)

Project: Shamsu — Browser API Factory & Marketplace

Document Version: 1.0

Status: Approved

Target Release: v3.15 (All Phases Complete)

Author: Shamsu Platform Team

1. Executive Summary & Value Proposition

Shamsu is a web-based API factory and marketplace designed for developers and non-technical business professionals. It solves the critical bottleneck of data scraping and automation setup by allowing users to turn standard browser operations (clicks, keystrokes, page scrapes) directly into functional, dynamic, and monetizable REST API endpoints.

Core Objective: Democratize browser-based web scraping, enabling anyone to create micro-service endpoints without writing any puppeteer scripting code manually, and allowing developers to purchase or subscribe to these APIs on a pay-per-call model.

2. User Roles & Journeys

A. The Scraper Creator (Seller)

- Journey:** Navigates to target website → Records interactions (clicks, text input, custom scrape target labels) → Stops recording → Receives AI-clarified API parameters and descriptions → Publishes API to marketplace with a set price (BDT) → Earns royalties from client subscriptions.

B. The API Client (Buyer)

- Journey:** Discovers APIs on the marketplace → Deposits money into wallet using integrated bKash gateway portal → Generates secure platform API keys → Sends REST POST requests to the API runner → Receives real-time scraped JSON payloads.

3. Product Requirements & Features

FEATURE 3.1 Zero-Extension Cloud AI Agent

Allows users to input a target website URL and a text goal (e.g. "Extract top stories and links"). The backend Puppeteer engine loads the DOM tree, cleans scripts/styles, and passes it to Groq Llama 3.3 / Gemini to automatically compile the target CSS selectors and interaction steps without requiring a Chrome extension install.

FEATURE 3.2 Dynamic Parameter Engine

The AI automatically converts hardcoded inputs typed during the recording sequence (such as city names or search inputs) into variable parameters (e.g., `{{search}}`). The REST API caller can swap these dynamically on request execution.

FEATURE 3.3 Integrated bKash Payment & Billing

Enforces pay-per-call monetisation. Consumers can deposit funds via standard bKash transaction IDs (TrxID). Admin verification triggers instant wallet balance updates. Platform charges BDT per call, distributing royalties directly to creators.

FEATURE 3.4 Automated Scheduler & Webhooks

Allows users to set cron timers (hourly/daily) for scraping. Integrates Llama 3.3 rule evaluation checks (e.g., "Check if price drops below 500 BDT") and triggers custom webhook alerts if the condition is met.

FEATURE 3.5 Developer Console & REST Sandbox

Equips creators with a postman-style request testing playground, displaying execution status codes, millisecond latencies, custom request header modifiers, and structured JSON syntax highlighting.

4. Non-Functional & Technical Specifications

- **Stealth Web Automation:** Puppeteer runner must run in stealth mode (User Agents, viewport spoofing, headless shell bypass) to bypass anti-scraping Cloudflare/captcha shields.
- **Self-Healing Selector Fallbacks:** If a target CSS selector breaks due to DOM adjustments, the backend automatically scans surrounding nodes for semantic text matches to execute clicks.
- **Database Consistency:** Neon PostgreSQL database must execute debounced state syncing to prevent database transaction bottlenecks during parallel runner requests.
- **Security and Isolation:** Restrict client calls to private APIs. Remove standard password hashes from admin endpoints.

5. Product Roadmap & Future Scope

- **Phase 1:** CAPTCHA and Cloudflare challenge auto-solvers integration.
- **Phase 2:** Multi-region rotating proxy configuration options inside Workspace.
- **Phase 3:** SDK Auto-generator supporting Python, Node.js, and Go client libraries.